



AVANCE DE PROYECTO



**MATERIA:
GRAFICACIÓN**

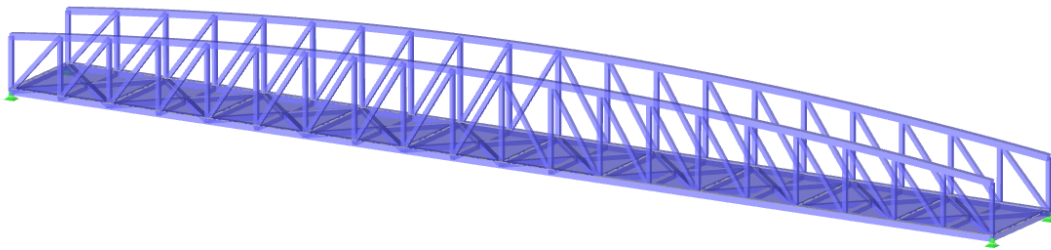
**NOMBRE DEL PROYECTO:
Simulador de deformación
estructural en un puente.**



**MAESTRO:
EURESTY URIBE CARLOS GERARDO
ALUMNOS:
-HERNÁNDEZ LÓPEZ CLARA PAULINA
-SERRANO SERRANO DIEGO**

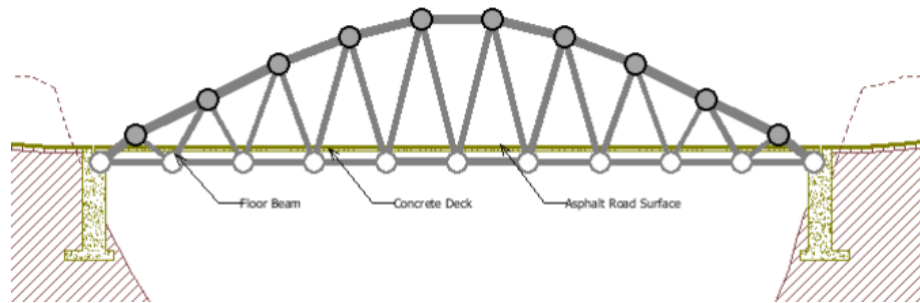
Objetivo General

Desarrollar un simulador grafico interactivo en 3D utilizando Python y OpenGL que permita visualizar y analizar la respuesta mecanica de un puente de armadura ante cargas dinamicas. El proposito es crear una herramienta educativa donde la computacion grafica sirva como puente para entender fenomenos fisicos complejos, como la deformacion elastica y la distribucion de esfuerzos en tiempo real.



Objetivos Especificos

- Implementar un motor de renderizado dinamico: configurar un entorno en OpenGL que no solo dibuje una estructura estatica, sino que permita la actualizacion constante de los vertices (nodos) en la GPU, permitiendo que el puente se deforme visualmente segun los calculos fisicos.
- Programar la logica de Cargas Dinamicas: desarrollar un algoritmo que simule el paso de un vehiculo sobre la estructura, transfiriendo su peso de forma proporcional entre los nodos a medida que se desplaza por el eje de la carretera.
- Codificar un sistema de visualizacion de esfuerzos (Stress Mapping): crear una funcion que traduzca las fuerzas internas de tension y compresion calculadas matematicamente en un gradiente cromatico (del azul al rojo).
- Simular la elasticidad y oscilacion: aplicar modelos matematicos (Ley de Hooke y amortiguamiento) para que el movimiento de la estructura presente vibraciones y deflexiones realistas ante el impacto de la carga.
- Establecer una interfaz interactiva basica: permitir que el usuario modifique variables en tiempo real, como la masa del vehiculo o la velocidad.



Justificacion

En la ingeniería civil y estructural, la capacidad de predecir como se comportara un puente antes de ser construido es vital para la seguridad publica. Segun Hibbeler (2012), el analisis dinamico permite identificar fenomenos que el analisis estatico ignora, como la fatiga de materiales y la resonancia. Este avance se enfoca precisamente en ese aspecto dinamico: ya no se trata solo de dibujar el puente, sino de hacerlo reaccionar correctamente al peso, al tiempo y a la velocidad de una carga real.

Visuales

Como senalan Shreiner et al. (2013) en la Guia Oficial de OpenGL, la representacion de datos mediante el color permite al cerebro humano procesar informacion mucho mas rapido que leyendo tablas de numeros. En este avance el gradiente de color ya no se inyecta de manera artificial, sino que se calcula directamente a partir del esfuerzo real de cada miembro segun la Ley de Hooke, lo que justifica el simulador como una verdadera herramienta de diagnostico visual.

Dificultad del proyecto

Desde el punto de vista de la materia de Graficacion, este avance se justifica por la complejidad de tener que sincronizar los calculos fisicos con el motor de renderizado en cada cuadro: el puente debe moverse segun la fisica, sin que el programa se trabe ni se cierre (Shreiner et al., 2013). La organizacion del codigo en un ciclo (loop) eficiente es un ejercicio de optimizacion practica que es fundamental en cualquier software moderno (Nystrom, 2014). El reto adicional de este avance es que ahora el ciclo no solo dibuja, tambien resuelve un sistema dinamico con sub-stepping para mantener estabilidad numerica.

Planteamiento del Problema

Para que el proyecto no sea solo un puente al azar, planteamos un problema real que se resuelve con la herramienta:

Imaginamos que tenemos un puente tipo armadura y queremos saber si es seguro que pase por ahi un camion de carga moderno, mucho mas pesado que los de antes. Actualmente es dificil visualizar donde sufre mas un puente cuando el peso se esta moviendo. Los calculos en papel

solo dicen el punto maximo de estres en un lugar fijo, pero no muestran el pandeo o la vibracion que ocurre mientras el vehiculo avanza.

- ¿En que parte exacta del trayecto el puente sufre mas presion?
- ¿Que vigas estan en riesgo de romperse si el camion va muy rapido?
- ¿Como cambia la deformacion si el material del puente es mas flexible o mas rigido?

Marco Teorico: Fundamentos Fisico-Matematicos

El analisis de una armadura Pratt en un entorno de tiempo real se fundamenta en la capacidad de resolver sistemas de ecuaciones diferenciales que describen el comportamiento elastico de los materiales frente a cargas externas. En este avance se implementan directamente los modelos descritos a continuacion.

1. Geometria y configuracion de la armadura

La armadura Pratt se define como un grafo ponderado donde los nodos son puntos de union y los miembros son aristas con propiedades de rigidez. La longitud inicial de cada elemento (L_0) se calcula con el teorema de Pitagoras:

$$L_0 = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

2. Modelo de elasticidad: Ley de Hooke (Sprint 3, Semana 5)

Cada viga se comporta como un resorte ultra-rigido. Si una carga externa mueve un nodo, la viga se estira o se encoge, generando una fuerza interna que intenta devolver el nodo a su lugar de reposo:

$$F = k * (L_{actual} - L_0), \quad k = E * A / L_0$$

Donde E es el Modulo de Young del material, A el area transversal y L_0 la longitud en reposo. Esta es la ecuacion que se ejecuta para cada miembro en cada sub-paso del simulador.

3. Modelo de carga dinamica e interpolacion (Semana 7)

El paso de un vehiculo se simula como una carga puntual $P = m * g$ que se desplaza sobre el eje x a una velocidad v. La posicion del vehiculo en el tiempo t es:

$$x(t) = x_{inicio} + v * t$$

Para evitar que la carga salte de un nodo a otro, se aplica una interpolacion lineal entre los dos nodos de la carretera mas cercanos. Si el auto esta entre el nodo k y el nodo k+1:

$$xi = (x(t) - x_k) / (x_{k+1} - x_k)$$

$$F_k = P * (1 - xi), \quad F_{k+1} = P * xi$$

4. Dinamica, oscilacion y amortiguamiento (Semana 8)

Para que el puente presente una vibracion realista se implementa la ecuacion de movimiento para sistemas con multiples grados de libertad:

$$M \cdot u'' + C \cdot u' + K \cdot u = F(t)$$

El termino $C \cdot u'$ es el amortiguamiento (Damping). Sin el, la estructura vibraría infinitamente como un resorte eterno. En el simulador se aplica una fuerza disipativa proporcional a la velocidad de cada nodo:

$$F_d = -c \cdot v$$

Esto modela la perdida de energia por friccion interna y resistencia del aire, y es lo que estabiliza la simulacion despues de cada impacto del vehiculo.

5. Analisis de esfuerzos y mapeo cromatico

Para la visualizacion en el motor grafico se calculan la deformacion unitaria y el esfuerzo de cada miembro:

$$\epsilon = (L_{actual} - L_0) / L_0, \quad \sigma = E \cdot \epsilon$$

- Si $\sigma > 0$ (tension): la barra se estira y se asigna color azul.
- Si $\sigma < 0$ (compresion): la barra se aplasta y se asigna color rojo.

6. Motor de tiempo: integracion de Verlet (Semana 6)

Para que la simulacion sea estable se sustituye la integracion de Euler del avance anterior por el algoritmo de Verlet, que conserva mejor la energia y es mas estable para sistemas oscilatorios. En cada sub-paso de tiempo dt , para cada nodo n :

- Calculo de fuerza total: $f = f_{vehiculo} + f_{elastica} + f_{amortiguamiento} + (m \cdot g)$.
- Segunda Ley de Newton: $a = f / m$.
- Actualizacion de Verlet: $x_{new} = 2 \cdot x - x_{prev} + a \cdot dt^2$.

La diferencia clave respecto a Euler es que Verlet no necesita guardar la velocidad como variable explicita: la velocidad queda implicita en la diferencia entre la posicion actual y la posicion previa, lo que reduce el error acumulado y previene que el puente explote por inestabilidad numerica.

Elaboracion del Proyecto

1. Modelado con diagramas UML

Diagrama de Clases (Arquitectura Orientada a Objetos)

Define la estructura estatica del simulador. Las entidades principales son:

- PuentePratt (Gestor central): contiene listas de Nodo y Miembro, ejecuta el ciclo de fisica y orquesta las fuerzas elasticas, gravedad, carga del vehiculo, amortiguamiento e integracion de Verlet.
- Nodo y Miembro (Entidades fisicas): el Nodo guarda posicion actual, posicion previa (para Verlet), masa y vector de fuerza acumulada. El Miembro guarda los dos nodos que conecta, su modulo de Young, su area, su longitud en reposo L_0 y su esfuerzo actual.

- CargaMóvil (Agente dinámico): mantiene su posición en el eje x, su masa y su velocidad. Se encarga de distribuir el peso entre los dos nodos de la carretera adyacentes mediante interpolación lineal.
- MotorGráfico (Capa de presentación): desacoplada de la física. Recibe el estado de los nodos y miembros y se encarga únicamente de los comandos de OpenGL para dibujar la estructura, los esfuerzos en color y el vehículo.

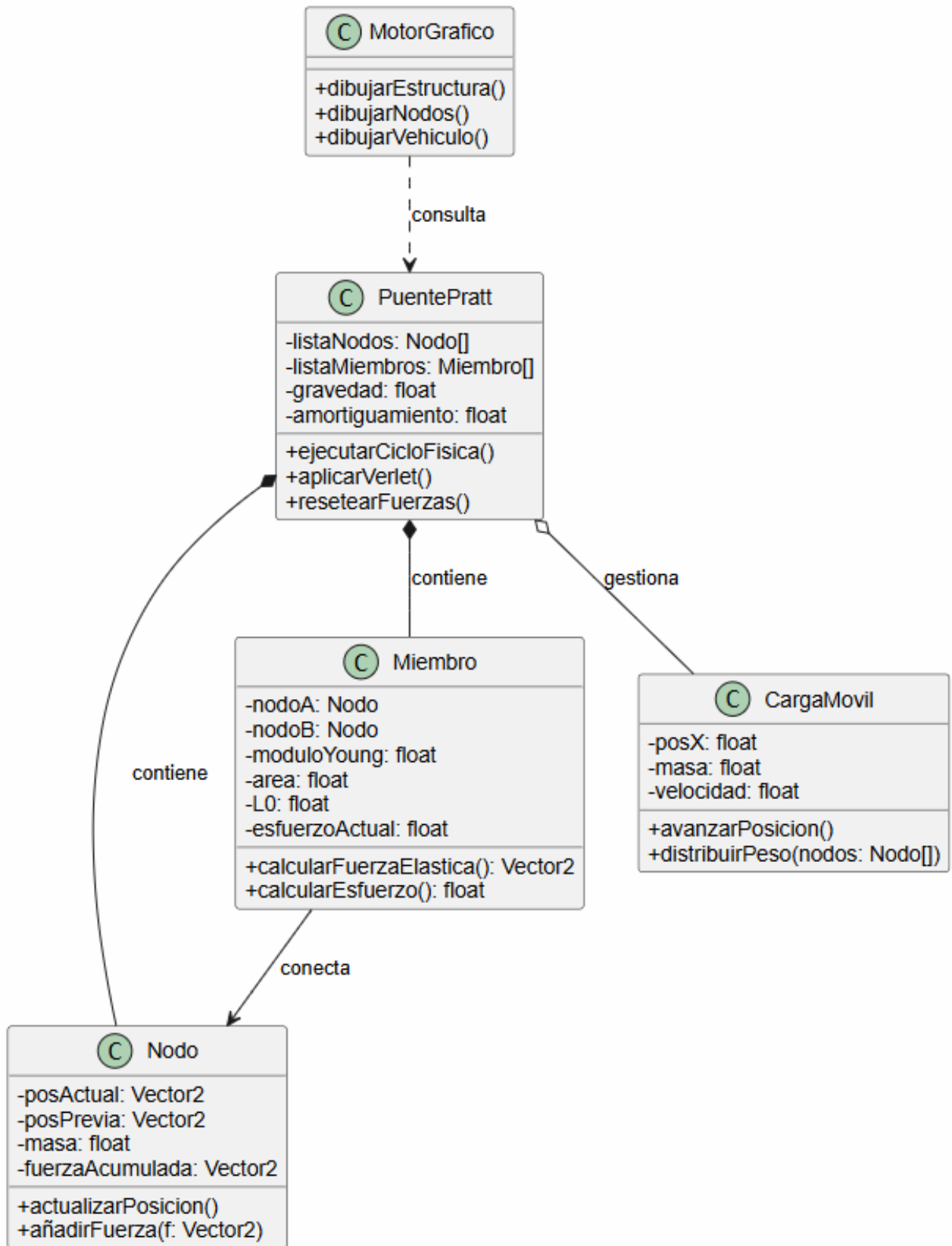


Diagrama de Casos de Uso (Interaccion)

Delimita las funcionalidades expuestas al actor principal (Usuario / Estudiante / Ingeniero):

- CU-01 Configurar Geometria Inicial: dimensiones, paneles y altura del puente.
- CU-02 Lanzar Vehiculo: el usuario presiona la barra espaciadora para iniciar el cruce del vehiculo sobre el puente.
- CU-03 Modificar Variables en Tiempo Real: el usuario altera la masa (flechas arriba/abajo) y la velocidad (flechas izquierda/derecha) sin necesidad de reiniciar la simulacion.
- CU-04 Visualizar Stress Mapping: el sistema traduce los datos reales de compresion/tension en colores rojo/azul proyectados sobre la estructura.
- CU-05 Observar Oscilacion y Amortiguamiento: tras el paso del vehiculo, el puente vibra y se estabiliza progresivamente gracias al damping.
- CU-06 Reiniciar (tecla R): regresa la estructura al estado de reposo total.

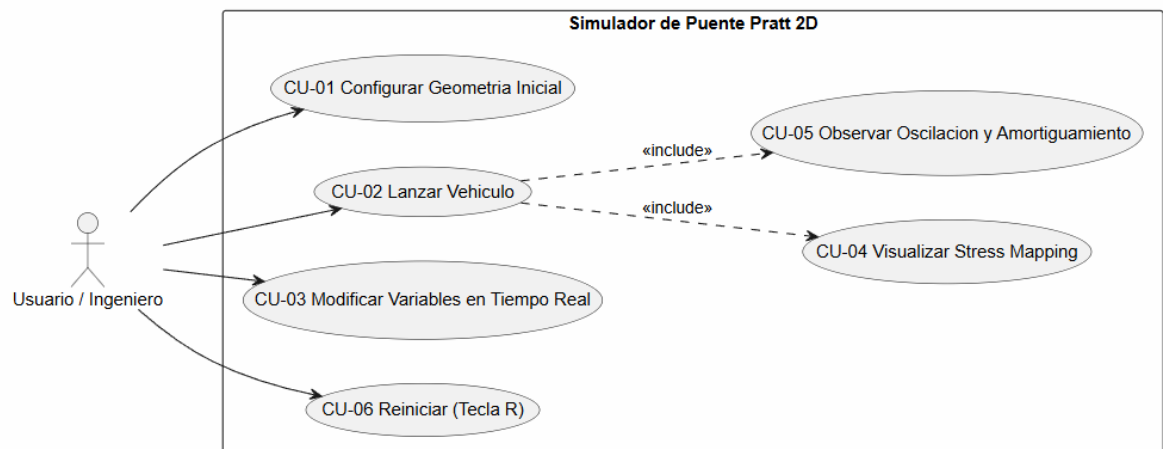
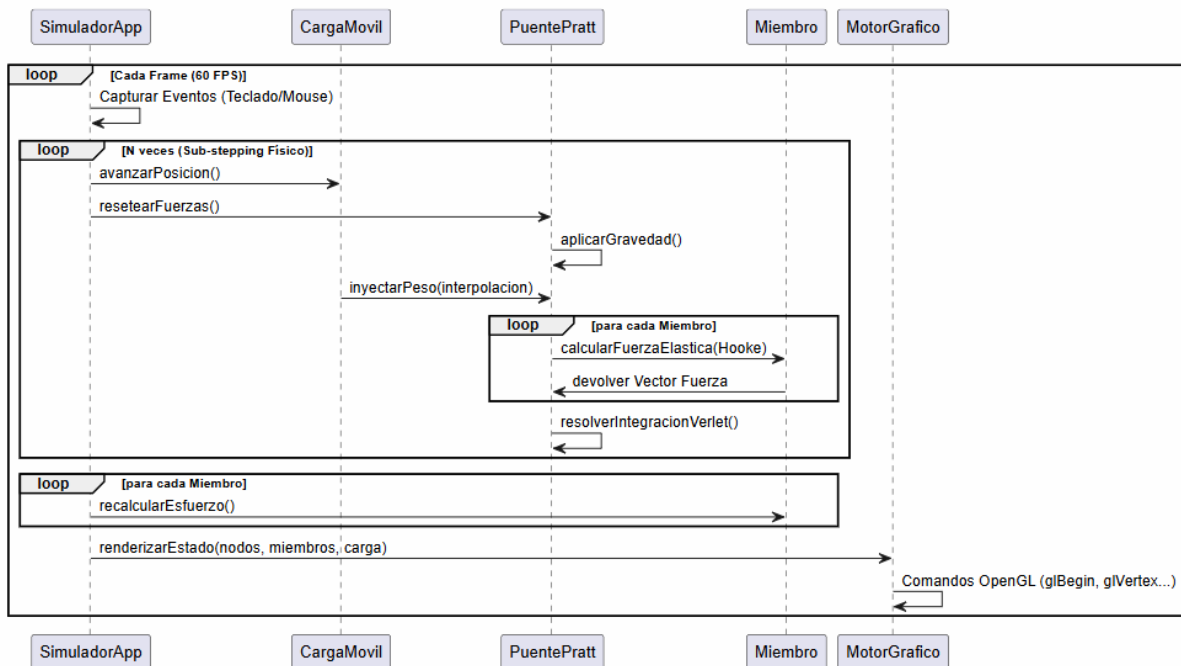


Diagrama de Secuencia (Game Loop)

Documenta el flujo temporal de un cuadro de simulacion a 60 FPS, con sub-stepping interno para estabilidad de Verlet:

1. Captura de eventos: SimuladorApp escucha entradas de Pygame (espacio, R, flechas).
2. Sub-paso fisico (se repite N veces por frame): CargaMovil avanza su posicion x; PuentePratt resetea fuerzas, aplica gravedad, inyecta el peso del vehiculo en los nodos cercanos por interpolacion, suma las fuerzas elasticas de Hooke, suma el amortiguamiento y resuelve la integracion de Verlet.
3. Calculo de esfuerzos: cada Miembro recalcula sigma a partir de la nueva longitud y determina su color.
4. Renderizado: MotorGrafico recibe el estado actualizado y emite los comandos de OpenGL para dibujar lineas, nodos y vehiculo. El ciclo vuelve a iniciar.



2. Algoritmos y librerías usadas

El stack se mantiene respecto al avance anterior, ahora con uso real de los módulos:

- pygame: esqueleto de la aplicación. Crea la ventana, gestiona el Game Loop a 60 FPS y captura los eventos del teclado para modificar masa y velocidad en tiempo real.
- PyOpenGL: motor gráfico. Dibuja los miembros con GL_LINES coloreados según el esfuerzo, los nodos con GL_POINTS y el vehículo con GL_QUADS. Permite la actualización de vértices cuadro a cuadro.
- numpy: cerebro matemático. Se usa para los vectores de fuerza acumulada de cada nodo, garantizando operaciones aritméticas eficientes.
- math: trigonometría y distancias (teorema de Pitágoras para L0, vectores unitarios para proyectar la fuerza de Hooke a lo largo de cada viga).
- sys: cierre limpio de la aplicación.

Algoritmos clave implementados en este avance:

- Ley de Hooke vectorial: $F = k * (L - L_0)$ proyectada sobre el vector unitario del miembro (Sprint 3, Semana 5).
- Integración de Verlet: $x_{\text{new}} = 2x - x_{\text{prev}} + a * dt^2$, con sub-stepping de 100 pasos por frame para garantizar estabilidad (Semana 6).
- Interpolación lineal de carga móvil entre los dos nodos adyacentes de la carretera (Semana 7).
- Amortiguamiento viscoso: $F_d = -c * v$, donde v se estima como $(x - x_{\text{prev}}) / dt$ (Semana 8).

3. Informacion sustentable (CSV, BD, Grafos)

La arquitectura de datos se mantiene en tres niveles: representacion en memoria (grafos), persistencia espacial (CSV) y catalogo de materiales (BD). En este avance la representacion como grafo se ejercita por completo: la matriz de fuerzas globales se construye implicitamente recorriendo las aristas (miembros) y acumulando su contribucion sobre los vertices (nodos). Esto valida la decision de modelar el puente como un grafo ponderado: la logica fisica es agnostica al numero de nodos y se podra escalar sin reescribir nada cuando se conecten los archivos CSV de geometria y la base de datos de materiales en los siguientes sprints.

4. Elaboracion de software sustentable (Metodologia Agil - Sprints)

El plan original Scrum se ha ejecutado en orden. El presente avance corresponde al cierre del Sprint 3, conforme al cronograma del anteproyecto:

- Sprint 1 - Cimientos y Geometria Estatica: COMPLETADO. Entorno Pygame/PyOpenGL, lectura de la topologia y dibujo del puente en reposo.
- Sprint 2 - Motor Estatico y preparacion matematica: COMPLETADO. Estructura de datos preparada para fuerzas y desplazamientos.
- Sprint 3 - Dinamica y Animacion (semanas 5-8): COMPLETADO en este avance. Incluye Ley de Hooke (semana 5), integracion de Verlet (semana 6), carga movil del vehiculo (semana 7) y amortiguamiento (semana 8).
- Sprint 4 - Stress Mapping refinado y pulido de UI: pendiente. Aunque el Stress Mapping ya esta operativo, en el siguiente avance se implementara con Fragment Shaders para mejor calidad visual y se agregara HUD con telemetria (estres maximo, masa, velocidad).

Cronograma de las semanas cubiertas en este avance:

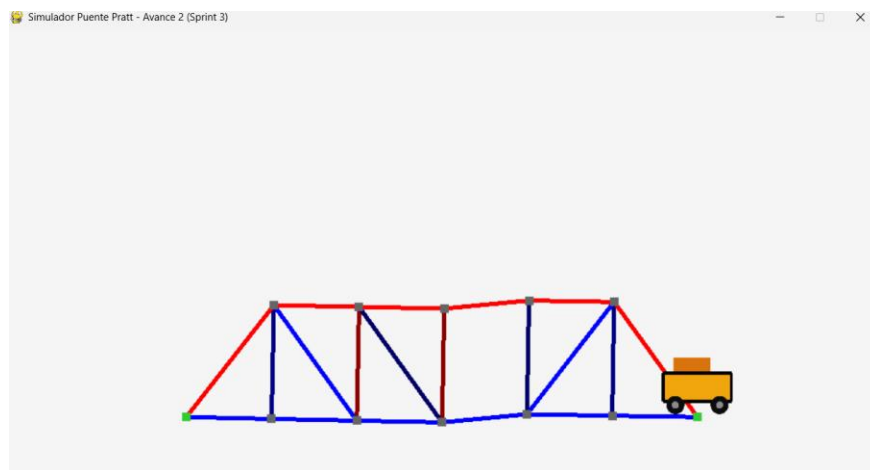
Semana	Fase / Actividad	Entregable Esperado
5	Modelo de Elasticidad: Programacion de la Ley de Hooke aplicada a cada viga basada en su modulo de Young.	Calculo de fuerzas internas.
6	Integracion Numerica: Implementacion del algoritmo de Verlet para el movimiento de los nodos.	Puente con gravedad (caida).
7	Carga Dinamica: Programacion de la masa movil (vehiculo) que recorre la via del puente.	Vehiculo cruzando el puente.
8	Amortiguamiento: Ajuste del Damping y estabilidad numerica para evitar oscilaciones irreales.	Segundo Avance: Simulacion Fisica.

Demostracion de su funcionamiento

El simulador entregado en este avance ya no es una prueba de concepto con datos aleatorios: cada color, cada vibración y cada deflexión provienen de un cálculo físico real. A continuación se describen los escenarios de prueba ejecutados.

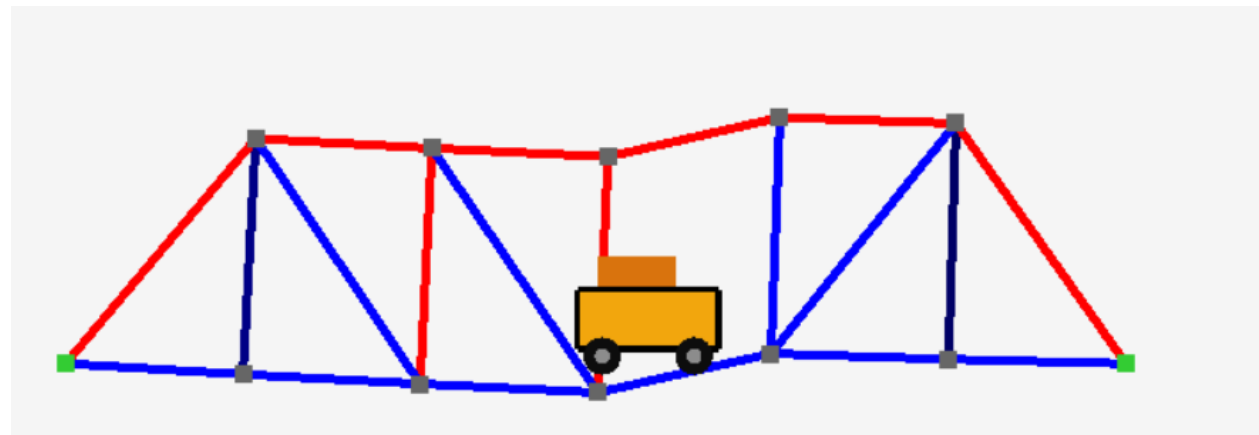
Estado de reposo bajo gravedad propia

Al iniciar la aplicación, los nodos no fijos caen ligeramente bajo su propio peso hasta que las fuerzas elásticas de los miembros equilibran a la gravedad. Las cuerdas inferiores quedan en tensión (azul tenue) y las verticales y diagonales superiores en compresión ligera (rojo tenue), tal como predice la teoría de armaduras de Hibbeler (2012). Los nodos verdes en los extremos son los apoyos fijos.



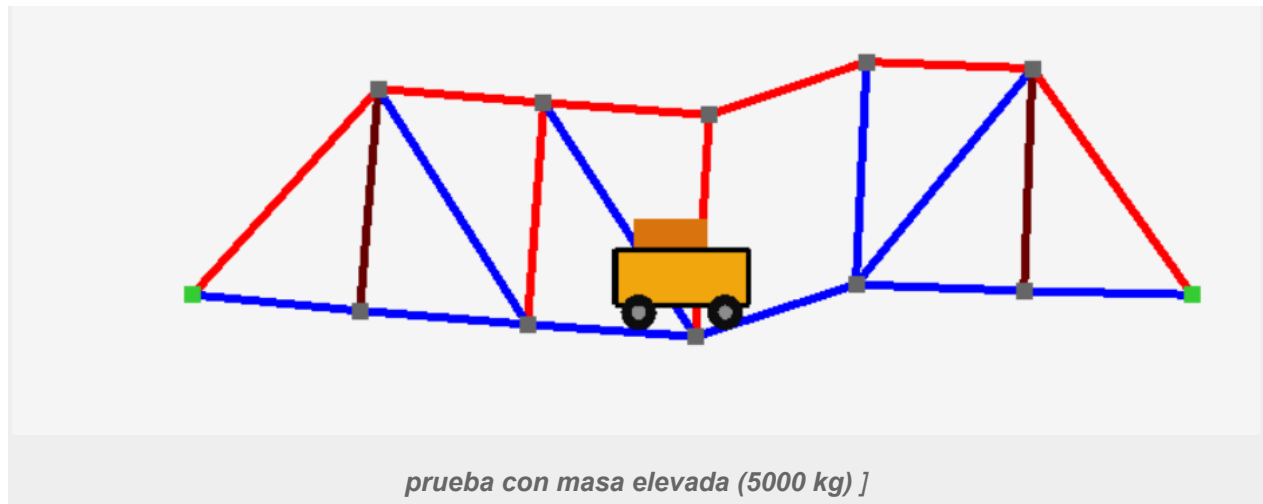
Cruce del vehículo (tecla ESPACIO)

Al presionar la barra espaciadora se lanza un vehículo de 2000 kg que recorre la carretera a 3 m/s. Durante el cruce se observa como el peso se distribuye correctamente entre los dos nodos adyacentes (interpolación lineal) y como la deflexión se concentra cerca de la posición instantánea del vehículo. Las cuerdas inferiores se intensifican en azul (tensión creciente) cuando el vehículo pasa sobre el centro del claro.



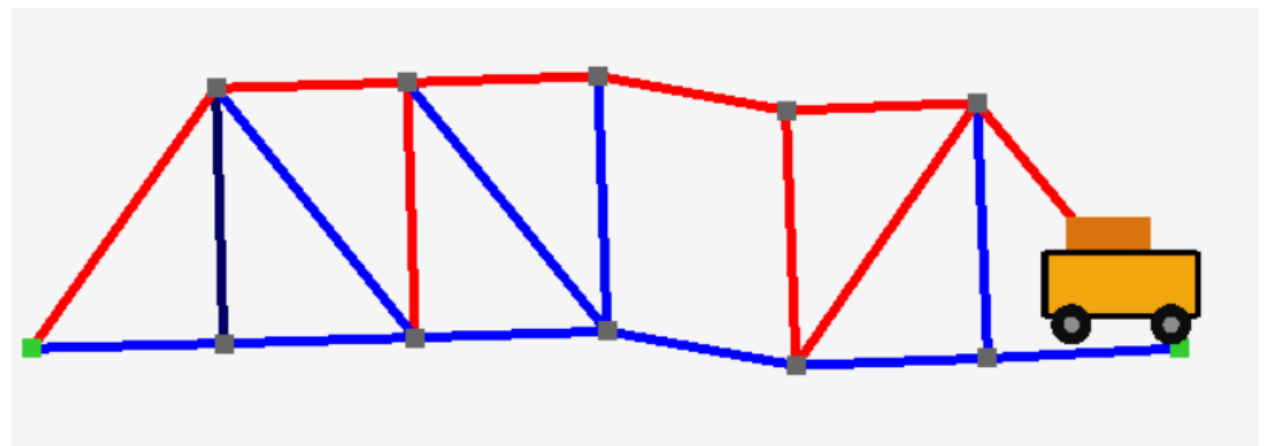
Modificación de variables en tiempo real

Las teclas de flechas permiten alterar masa (arriba/abajo) y velocidad (derecha/izquierda) sin reiniciar la simulación. Al aumentar la masa, la deflexión crece proporcionalmente y las barras críticas saturan su color, validando que el sistema responde a la física. Al aumentar la velocidad, se observa como el puente apenas alcanza a equilibrarse antes de que la carga se mueva al siguiente panel, mostrando el efecto dinámico que justifica el proyecto.



Oscilacion amortiguada post-cruce

Una vez que el vehículo abandona el puente, la estructura sigue oscilando libremente, pero la amplitud disminuye en cada ciclo gracias al término $F_d = -c \cdot v$. En pocos segundos el puente regresa al estado de equilibrio bajo su gravedad propia. Esta es la verificación directa del Sprint 4 (Semana 8): si el coeficiente c fuera cero, el puente vibraría indefinidamente.



Vision del Prototipo Final

Lo entregado en este avance constituye la columna vertebral del simulador. La vision para los avances restantes y la entrega final consiste en cuatro frentes:

- Stress Mapping con shaders: trasladar el calculo de color del CPU al Fragment Shader de OpenGL para liberar carga en la CPU y mejorar la calidad visual con un gradiente continuo en lugar de tres niveles discretos.
- Carga de geometria desde CSV: externalizar nodos.csv y miembros.csv para poder probar otras armaduras (Howe, Warren) sin tocar el codigo fuente.
- Catalogo de materiales en SQLite: una base de datos con Modulo de Young y limite de rotura para acero, aluminio, madera y concreto, seleccionable en tiempo real.
- HUD informativo: panel sobreimpreso que muestre masa, velocidad, esfuerzo maximo actual y nodo critico, dirigido al uso educativo.
- Modo de fallo: cuando una barra supera su limite de rotura, marcarla con color saturado y emitir una alerta visual.

Analisis de los Resultados

Los resultados de este avance validan los objetivos planteados para el Sprint 3:

- Estabilidad numerica: el sub-stepping de 100 pasos por frame combinado con Verlet permite mantener 60 FPS sin que la estructura explote, incluso con cargas elevadas. El cambio de Euler a Verlet fue determinante para evitar el efecto de resorte eterno y preservar la energia.
- Realismo dinamico: la deflexion no es ya un valor inyectado manualmente, sino la solucion en tiempo real de un sistema masa-resorte-amortiguador. Esto cumple con el objetivo justificado en el marco teorico de Chopra (2014) de observar fenomenos dinamicos imposibles de capturar con un analisis estatico.
- Diagnostico visual efectivo: los colores corresponden ahora al esfuerzo real $\sigma = E \cdot \epsilon$. Es posible identificar a simple vista cuales son las cuerdas inferiores en tension y cuales las diagonales en compresion, validando el Stress Mapping como herramienta pedagogica.
- Interactividad responsiva: los cambios en masa y velocidad se reflejan instantaneamente en la deformacion y en los colores, sin necesidad de reiniciar el ciclo principal. Esto valida el Caso de Uso CU-03.
- Limitaciones detectadas: la rigidez $E = 2e8$ Pa esta escalada respecto al acero real (200 GPa) para que la deflexion sea visible a la escala del puente. Esta sera una de las primeras parametrizaciones que se enviaron a base de datos en el Sprint 4.

Referencias

- Beer, F. P., Johnston, E. R., & DeWolf, J. T. (2017). Mecanica de Materiales. McGraw-Hill.
- Chopra, A. K. (2014). Dinamica de Estructuras. Pearson Educacion.
- Hibbeler, R. C. (2012). Analisis Estructural. Pearson Educacion.
- Nystrom, R. (2014). Game Programming Patterns. Genever Benning.
- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL. Addison-Wesley.
- Witkin, A., & Baraff, D. (2001). Physically Based Modeling. SIGGRAPH Course Notes.
- Documentacion oficial de Pygame: <https://www.pygame.org/docs/>
- Documentacion oficial de PyOpenGL: <http://pyopengl.sourceforge.net/documentation/>
- Documentacion oficial de NumPy: <https://numpy.org/doc/>

Anexos:Codigo

A continuacion se incluye el codigo fuente completo del simulador correspondiente al cierre del Sprint 3. Implementa: Ley de Hooke (semana 5), integracion de Verlet (semana 6), carga movil con interpolacion lineal (semana 7) y amortiguamiento viscoso (semana 8).

Controles de teclado:

- ESPACIO: lanzar vehiculo de izquierda a derecha.
- R: reiniciar la simulacion al estado de reposo.
- Flecha arriba / abajo: aumentar / disminuir masa del vehiculo (en pasos de 500 kg).
- Flecha derecha / izquierda: aumentar / disminuir velocidad del vehiculo (en pasos de 1 m/s).

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *
import numpy as np
import math
import sys

GRAVEDAD = 9.81
SUBSTEPS = 80
class Nodo:
    def __init__(self, id_nodo, x, y, fijo=False, masa=80.0):
        self.id = id_nodo
        self.x0 = x
        self.y0 = y
        self.x_eq = x
        self.y_eq = y
        self.x = x
        self.y = y
        self.x_prev = x
        self.y_prev = y
        self.fijo = fijo
        self.masa = masa
        self.fuerza = np.array([0.0, 0.0])

    @property
    def dx(self):
        return self.x - self.x_eq

    @property
    def dy(self):
        return self.y - self.y_eq

    @property
    def pos_actual(self):
        return (self.x, self.y)

    def reset_fuerza(self):
        self.fuerza[0] = 0.0
        self.fuerza[1] = 0.0

    def aplicar_fuerza(self, fx, fy):
```

```
self.fuerza[0] += fx
self.fuerza[1] += fy
```

```
class Miembro:
```

```
def __init__(self, id_miembro, nodo_origen, nodo_destino, E=1.0e9, area=0.01):
    self.id = id_miembro
    self.origen = nodo_origen
    self.destino = nodo_destino
    self.E = E
    self.Area = area
    self.esfuerzo_actual = 0.0
```

```
    dx = self.destino.x0 - self.origen.x0
    dy = self.destino.y0 - self.origen.y0
    self.L0 = math.hypot(dx, dy)
    self.k = self.E * self.Area / self.L0
```

```
def longitud_actual(self):
    dx = self.destino.x - self.origen.x
    dy = self.destino.y - self.origen.y
    return math.hypot(dx, dy)
```

```
def calcular_esfuerzo(self):
    L = self.longitud_actual()
    epsilon = (L - self.L0) / self.L0
    self.esfuerzo_actual = self.E * epsilon
    return self.esfuerzo_actual
```

```
def aplicar_fuerza_hooke(self):
    L = self.longitud_actual()
    if L < 1e-9:
        return
    delta_L = L - self.L0
    F = self.k * delta_L
    ux = (self.destino.x - self.origen.x) / L
    uy = (self.destino.y - self.origen.y) / L
    fx = F * ux
    fy = F * uy
    self.origen.aplicar_fuerza(fx, fy)
    self.destino.aplicar_fuerza(-fx, -fy)
```

```
def obtener_color_stress(self):
    intensidad = min(abs(self.esfuerzo_actual) / umbral, 1.0)
    if self.esfuerzo_actual > 1000:      # Tension
        return (0.0, 0.0, max(0.4, intensidad))
    elif self.esfuerzo_actual < -1000:   # Compresion
        return (max(0.4, intensidad), 0.0, 0.0)
    return (0.2, 0.2, 0.2)
```

```
class CargaMovil:
```

```
def __init__(self, masa=2000.0, velocidad=3.0):
    self.masa = masa
    self.velocidad = velocidad      # m/s
    self.pos_x = 0.0
    self.activo = False
    self.x_min = 0.0
    self.x_max = 0.0
```

```
def configurar_recorrido(self, x_min, x_max):
```

```

        self.x_min = x_min
        self.x_max = x_max
        self.pos_x = x_min

    def iniciar(self):
        self.pos_x = self.x_min
        self.activo = True

    def detener(self):
        self.activo = False
        self.pos_x = self.x_min

    def actualizar(self, dt):
        if not self.activo:
            return
        self.pos_x += self.velocidad * dt
        if self.pos_x > self.x_max:
            self.pos_x = self.x_max
            self.activo = False

    def aplicar_carga(self, nodos_carretera):
        if not self.activo:
            return
        if self.pos_x < nodos_carretera[0].x0 or self.pos_x > nodos_carretera[-1].x0:
            return
        peso = self.masa * GRAVEDAD
        for i in range(len(nodos_carretera) - 1):
            nk = nodos_carretera[i]
            nk1 = nodos_carretera[i + 1]
            if nk.x0 <= self.pos_x <= nk1.x0:
                xi = (self.pos_x - nk.x0) / (nk1.x0 - nk.x0)
                Fk = peso * (1.0 - xi)
                Fk1 = peso * xi
                nk.aplicar_fuerza(0.0, -Fk)
                nk1.aplicar_fuerza(0.0, -Fk1)
                break

    def get_pos_y(self, nodos_carretera):
        if self.pos_x <= nodos_carretera[0].x0:
            return nodos_carretera[0].y
        if self.pos_x >= nodos_carretera[-1].x0:
            return nodos_carretera[-1].y
        for i in range(len(nodos_carretera) - 1):
            nk = nodos_carretera[i]
            nk1 = nodos_carretera[i + 1]
            if nk.x0 <= self.pos_x <= nk1.x0:
                xi = (self.pos_x - nk.x0) / (nk1.x0 - nk.x0)
                return nk.y * (1.0 - xi) + nk1.y * xi
        return 0.0

class PuentePratt:
    def __init__(self):
        self.nodos = []
        self.miembros = []
        self.nodos_carretera = []
        self.amortiguamiento = 5000.0

    def generar_parametrizado(self, paneles, longitud_panel, altura):
        self.nodos.clear()
        self.miembros.clear()
        self.nodos_carretera.clear()

```

```

    nodos_inf = []
    for i in range(paneles + 1):
        es_apoyo = (i == 0 or i == paneles)
        nodo = Nodo(len(self.nodos), i * longitud_panel, 0.0,
                    fijo=es_apoyo, masa=80.0)
        self.nodos.append(nodo)
        nodos_inf.append(nodo)
        self.nodos_carretera.append(nodo)

    nodos_sup = []
    for i in range(1, paneles):
        nodo = Nodo(len(self.nodos), i * longitud_panel, altura, masa=40.0)
        self.nodos.append(nodo)
        nodos_sup.append(nodo)

    id_m = 0
    def conectar(n1, n2):
        nonlocal id_m
        self.miembros.append(Miembro(id_m, n1, n2))
        id_m += 1

    for i in range(len(nodos_inf) - 1):
        conectar(nodos_inf[i], nodos_inf[i + 1])
    for i in range(len(nodos_sup) - 1):
        conectar(nodos_sup[i], nodos_sup[i + 1])
    # Verticales
    for i in range(len(nodos_sup)):
        conectar(nodos_inf[i + 1], nodos_sup[i])
    # Verticales de Los extremos
    conectar(nodos_inf[0], nodos_sup[0])
    conectar(nodos_inf[-1], nodos_sup[-1])
    mitad = paneles // 2
    for i in range(1, paneles - 1):
        if i < mitad:
            conectar(nodos_sup[i - 1], nodos_inf[i + 1])
        elif i > mitad:
            conectar(nodos_inf[i], nodos_sup[i])

    def reset_a_equilibrio(self):
        for n in self.nodos:
            n.x = n.x_eq
            n.y = n.y_eq
            n.x_prev = n.x_eq
            n.y_prev = n.y_eq
            n.fuerza[:] = 0.0
        for m in self.miembros:
            m.calcular_esfuerzo()

    def paso_fisico(self, dt, carga_movil=None, aplicar_gravedad=True):
        for n in self.nodos:
            n.reset_fuerza()

        if aplicar_gravedad:
            for n in self.nodos:
                if not n.fijo:
                    n.aplicar_fuerza(0.0, -n.masa * GRAVEDAD)

        if carga_movil:
            carga_movil.aplicar_carga(self.nodos_carretera)

```

```

for m in self.miembros:
    m.aplicar_fuerza_hooke()

for n in self.nodos:
    if not n.fijo:
        vx = (n.x - n.x_prev) / dt
        vy = (n.y - n.y_prev) / dt
        n.aplicar_fuerza(-self.amortiguamiento * vx,
                        -self.amortiguamiento * vy)

for n in self.nodos:
    if n.fijo:
        continue
    ax = n.fuerza[0] / n.masa
    ay = n.fuerza[1] / n.masa
    x_new = 2.0 * n.x - n.x_prev + ax * dt * dt
    y_new = 2.0 * n.y - n.y_prev + ay * dt * dt
    n.x_prev = n.x
    n.y_prev = n.y
    n.x = x_new
    n.y = y_new

for m in self.miembros:
    m.calcular_esfuerzo()

def relajar_a_equilibrio(self, n_iter=6000, dt=0.0015):
    damp_orig = self.amortiguamiento
    self.amortiguamiento = 18000.0
    for _ in range(n_iter):
        self.paso_fisico(dt, carga_movil=None, aplicar_gravedad=True)
    for n in self.nodos:
        n.x_eq = n.x
        n.y_eq = n.y
        n.x_prev = n.x
        n.y_prev = n.y
    self.amortiguamiento = damp_orig

class MotorGrafico:
    def __init__(self, puente):
        self.puente = puente

    def dibujar(self, carga_movil=None):
        glEnable(GL_LINE_SMOOTH)
        glHint(GL_LINE_SMOOTH_HINT, GL_NICEST)
        glLineWidth(4.0)

        glBegin(GL_LINES)
        for m in self.puente.miembros:
            color = m.obtener_color_stress()
            glColor3f(*color)
            x1, y1 = m.origen.pos_actual
            x2, y2 = m.destino.pos_actual
            glVertex3f(x1, y1, 0)
            glVertex3f(x2, y2, 0)
        glEnd()

        glPointSize(10.0)
        glBegin(GL_POINTS)
        for n in self.puente.nodos:
            if n.fijo:

```

```

        glColor3f(0.2, 0.8, 0.2)
    else:
        glColor3f(0.4, 0.4, 0.4)
        glVertex3f(*n.pos_actual, 0)
    glEnd()

    if carga_movil is not None:
        y_v = carga_movil.get_pos_y(self.puente.nodos_carretera)
        self._dibujar_vehiculo(carga_movil.pos_x, y_v + 0.55)

def _dibujar_vehiculo(self, x, y):
    ancho, alto = 2.4, 1.0
    glColor3f(0.95, 0.65, 0.05)
    glBegin(GL_QUADS)
    glVertex3f(x - ancho/2, y, 0)
    glVertex3f(x + ancho/2, y, 0)
    glVertex3f(x + ancho/2, y + alto, 0)
    glVertex3f(x - ancho/2, y + alto, 0)
    glEnd()

    glColor3f(0.85, 0.45, 0.05)
    cab_w = ancho * 0.55
    cab_h = alto * 0.55
    glBegin(GL_QUADS)
    glVertex3f(x - cab_w/2 - 0.2, y + alto, 0)
    glVertex3f(x + cab_w/2 - 0.2, y + alto, 0)
    glVertex3f(x + cab_w/2 - 0.2, y + alto + cab_h, 0)
    glVertex3f(x - cab_w/2 - 0.2, y + alto + cab_h, 0)
    glEnd()

    glColor3f(0.0, 0.0, 0.0)
    glLineWidth(2.5)
    glBegin(GL_LINE_LOOP)
    glVertex3f(x - ancho/2, y, 0)
    glVertex3f(x + ancho/2, y, 0)
    glVertex3f(x + ancho/2, y + alto, 0)
    glVertex3f(x - ancho/2, y + alto, 0)
    glEnd()

    rueda_r = 0.32
    rueda_y = y - rueda_r * 0.4
    for cx in (x - ancho/2 * 0.65, x + ancho/2 * 0.65):
        self._dibujar_circulo(cx, rueda_y, rueda_r, (0.05, 0.05, 0.05))
        self._dibujar_circulo(cx, rueda_y, rueda_r * 0.4, (0.55, 0.55, 0.55))

def _dibujar_circulo(self, cx, cy, r, color, segs=24):
    glColor3f(*color)
    glBegin(GL_TRIANGLE_FAN)
    glVertex3f(cx, cy, 0)
    for i in range(segs + 1):
        th = 2.0 * math.pi * i / segs
        glVertex3f(cx + r * math.cos(th), cy + r * math.sin(th), 0)
    glEnd()

class SimuladorApp:
    def __init__(self):
        pygame.init()
        self.display = (1024, 768)
        pygame.display.set_mode(self.display, DOUBLEBUF | OPENGL)
        pygame.display.set_caption("Simulador Puente Pratt - Avance 2 (Sprint 3)")

```

```

self.paneles = 6
self.long_panel = 3.0
self.altura = 4.0

self.puente = PuentePratt()
self.puente.generar_parametrizado(self.paneles, self.long_panel, self.altura)

print("Calculando equilibrio estatico bajo peso propio...")
self.puente.relajar_a_equilibrio()
print("Equilibrio alcanzado. Listo para simular.")

self.motor_grafico = MotorGrafico(self.puente)

self.carga_movil = CargaMovil(masa=2000.0, velocidad=3.0)
self.carga_movil.configurar_recorrido(0.0, self.paneles * self.long_panel)

self.cam_x = -9.0
self.cam_y = -2.0
self.cam_z = -28.0
self.zoom = 45.0

self._imprimir_controles()

def _imprimir_controles(self):
    print("\n=== Controles ===")
    print("ESPACIO      : Lanzar vehiculo")
    print("R              : Reiniciar al equilibrio")
    print("Flecha ARRIBA   : +500 kg al vehiculo")
    print("Flecha ABAJO    : -500 kg al vehiculo")
    print("Flecha DERECHA  : +1 m/s velocidad")
    print("Flecha IZQUIERDA: -1 m/s velocidad")
    print()

def configurar_proyeccion(self):
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluPerspective(self.zoom, (self.display[0] / self.display[1]), 0.1, 100.0)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    glTranslatef(self.cam_x, self.cam_y, self.cam_z)

def manejar_eventos(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE:
                print(f"Vehiculo lanzado: m={self.carga_movil.masa:.0f} kg, "
                      f"v={self.carga_movil.velocidad:.1f} m/s")
                self.carga_movil.iniciar()
            elif event.key == pygame.K_r:
                print("Reiniciando al equilibrio.")
                self.carga_movil.detener()
                self.puente.reset_a_equilibrio()
            elif event.key == pygame.K_UP:
                self.carga_movil.masa += 500.0
                print(f"Masa vehiculo: {self.carga_movil.masa:.0f} kg")
            elif event.key == pygame.K_DOWN:
                self.carga_movil.masa = max(500.0, self.carga_movil.masa - 500.0)

```

```

        print(f"Masa vehiculo: {self.carga_movil.masa:.0f} kg")
    elif event.key == pygame.K_RIGHT:
        self.carga_movil.velocidad += 1.0
        print(f"Velocidad: {self.carga_movil.velocidad:.1f} m/s")
    elif event.key == pygame.K_LEFT:
        self.carga_movil.velocidad = max(1.0,
                                          self.carga_movil.velocidad - 1.0)
        print(f"Velocidad: {self.carga_movil.velocidad:.1f} m/s")

def ejecutar(self):
    clock = pygame.time.Clock()
    glClearColor(0.96, 0.96, 0.96, 1.0)

    while True:
        frame_dt = clock.tick(60) / 1000.0
        frame_dt = min(frame_dt, 1.0 / 30.0)

        self.manejar_eventos()

        sub_dt = frame_dt / SUBSTEPS
        for _ in range(SUBSTEPS):
            self.carga_movil.actualizar(sub_dt)
            self.puente.paso_fisico(sub_dt, self.carga_movil,
                                   aplicar_gravedad=True)

        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
        self.configurar_proyeccion()
        self.motor_grafico.dibujar(self.carga_movil)
        pygame.display.flip()

if __name__ == "__main__":
    app = SimuladorApp()
    app.ejecutar()

```